

SHOE XPRESS

Inventory Intelligence

How the whole system works, in plain language

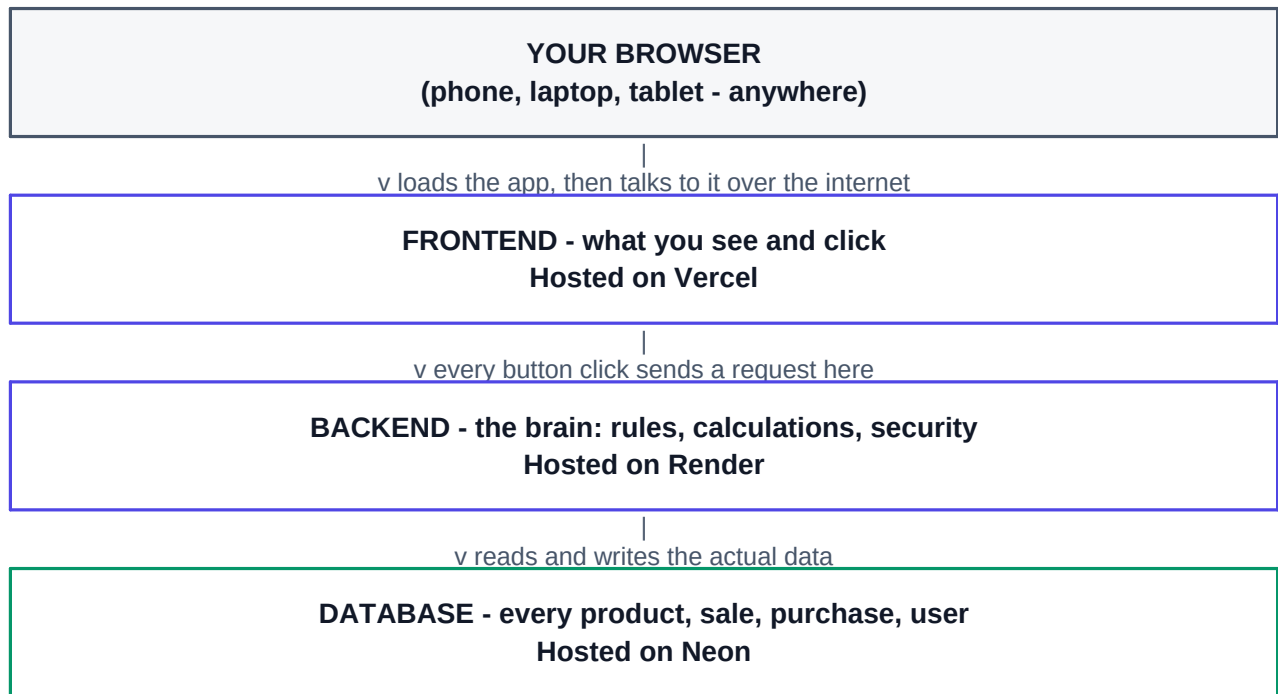
Generated 06 September 2026

This document explains - without assuming you're a programmer - where every piece of this system physically lives on the internet, how a click on your screen turns into a database update, how the 'AI' forecasting actually works under the hood, and the handful of internal rules that matter most if something ever looks wrong.

Nothing here is marketing language. Where something is a limitation or a cost you should know about, it's called out plainly.

1. The Big Picture

The system is split into three independent pieces, each hosted on a different specialist platform, talking to each other over the internet. This is completely standard practice for modern web software - each platform does the one job it's best at.



The important idea to take away: **your browser never talks to the database directly**. Every single number you see on screen was calculated by the backend, from data in the database, at the moment you asked for it. There is no separate 'report generator' or spreadsheet lurking somewhere with different numbers - there is exactly one source of truth.

Where each piece actually lives

Frontend (what you see)	React app, hosted on Vercel , at <code>inventory-management-wheat-two.vercel.app</code> . This is just the visual interface - buttons, tables, charts. It has no logic of its own; it asks the backend for everything.
Backend (the brain)	FastAPI (Python), hosted on Render , at <code>shoexpress-api.onrender.com</code> . Every calculation - stock levels, forecasts, recommendations, permissions - happens here, never in your browser.
Database (the data)	PostgreSQL, hosted on Neon (a managed Postgres provider). This is a plain, industry-standard relational database - the same kind of technology banks and e-commerce sites use. It is <i>not</i> a spreadsheet and not editable by hand; only the backend writes to it.

Source code

Stored on **GitHub** (deveshgoel02/Inventory-Management), privately. Every change either of us makes goes there first, and both Vercel and Render watch that repository and **automatically redeploy** within about a minute of any update - no manual 'publish' step.

2. What Actually Happens When You Click a Button

Take a concrete example: you open the Inventory page. Here is the literal chain of events, in order:

- Your browser fetches the page's code from **Vercel** - this happens once, fast, and is just files (HTML/JS/CSS), no business data.
- The page immediately makes a request to the **Render** backend: 'give me the current stock list.' This request includes a security token proving who you are (see Section 5).
- The backend checks that token, checks your role has permission to view inventory, then runs a query against the **Neon** database.
- Critically, the backend does **not** just read a stored 'current stock' number. It adds up every single stock transaction ever recorded for each product - every purchase, every sale, every return, every manual adjustment - to compute the true current quantity, live, every time. This is explained fully in Section 4.
- The backend sends the computed numbers back as data (JSON). The frontend then draws the table and charts you see. No calculation happens in the browser - it only displays what the backend already worked out.

The same pattern repeats for literally every page: dashboard, sales, purchasing, forecasting, recommendations. The frontend is a display layer; the backend is authoritative.

Why it's built this way

If financial and inventory numbers were calculated in the browser (in JavaScript), two different people could see two different 'correct' answers depending on their device, or a user could tamper with the page to see fake numbers. Keeping all authoritative math on the server closes that door entirely - the backend is the only thing that can be trusted, and it's the only thing that touches the database.

3. Hosting Details - What's Free, What Isn't, and the One Catch

Vercel (frontend)	Free tier. Effectively unlimited for this scale of use. No known limitations for how this app uses it.
Render (backend)	Free tier. The one real catch: a free Render service goes to sleep after about 15 minutes of no traffic. The next request after that has to wait 30-60 seconds while it wakes back up - you'll see a slightly longer delay than usual on the first click of the day. After that it's fast until it goes idle again. Upgrading to a paid Render plan (a small monthly fee) removes this delay entirely if it becomes annoying.
Neon (database)	Free tier: 0.5 GB storage, which comfortably covers years of a business at this scale before it becomes a concern. It also 'scales to zero' when unused, which can add a similar brief wake-up delay, same as Render.
GitHub (code storage)	Free, private repository. This is not customer-facing at all - it's where the actual program code is kept and version-tracked.

Total running cost today: **Rs. 0/month**. The trade-off for that is the wake-up delay described above - acceptable for a small team checking the system a handful of times a day, worth revisiting if usage grows heavily or becomes customer-facing.

4. The Core Idea: Stock Is Never Just 'A Number'

This is the single most important internal rule in the whole system, and the one most worth understanding even if nothing else in this document sticks.

There is no field anywhere in the database called 'current stock' that gets edited directly. Instead, every single event that changes stock - a purchase received, a sale made, a customer return, a manual correction - is written down permanently as one row in something like a bank statement (internally called the **inventory ledger**). Current stock for any product is always: *add up every one of those rows for that product, right now.*

The nine kinds of events that can move stock

- **Opening Balance** - the starting quantity when a SKU first enters the system (e.g. from an import).
- **Purchase** - stock received from a supplier (adds).
- **Sale** - stock sold to a customer (subtracts).
- **Sale Return** - a customer sent something back (adds).
- **Purchase Return** - stock sent back to a supplier (subtracts).
- **Stock Adjustment In / Out** - manual corrections, e.g. damaged goods found, or a stock-take correction (adds or subtracts, always with a required note explaining why).
- **Transfer In / Out** - stock moved between warehouses.

The practical benefit: every unit of stock can always be traced back to exactly which transaction put it there or took it away. If a number ever looks wrong, the fix is never 'overwrite the number' - it's 'look at the ledger for that SKU and find the transaction that's wrong,' which is always possible because nothing is ever silently deleted.

One safety rule enforced by the backend: the system will refuse to record a sale that would push stock below zero (unless it's explicitly flagged as historical data being imported, where old records sometimes have gaps). This is a real, tested guardrail, not just a suggestion.

5. Logging In and Permissions - How Security Works

When you log in, the backend checks your email and password, and if correct, hands your browser a signed, tamper-proof digital token (a JWT - 'JSON Web Token'). Your browser stores that token and attaches it to every request it makes afterwards, like a stamped ID card. The backend checks that stamp on every single request - nobody can ask for data or make a change without a valid one, and it expires automatically after 8 hours, at which point you'd need to log in again.

Four roles, and what each can do

ADMIN	Everything, including creating/editing other users and changing anyone's role.
MANAGER	Everything operational - sales, purchasing, inventory, imports, AI settings - except managing other users' accounts.
INVENTORY_STAFF	Can view and adjust stock, record sales and receipts, run imports - day-to-day warehouse work, no configuration access.
VIEWER	Read-only access to everything, plus the ability to export reports. Can't change anything.

Important: these checks happen on the backend, not just by hiding buttons in the interface. Even if someone found a way to make a request directly (bypassing the visible app entirely), the backend would still refuse it if their role doesn't have permission. There's also a specific safeguard preventing the very last remaining ADMIN account from being demoted or deactivated by anyone - including themselves - so the system can never end up with nobody able to manage users.

6. The 'AI' - What It Actually Is (and Isn't)

This is worth being very precise about, because 'AI' gets used loosely everywhere. There is **no ChatGPT-style language model** involved in any number this system produces, and no neural network making purchase decisions. What it actually uses is real, well-established statistics - the same kind of forecasting math used in supply-chain and demand-planning software for decades. That's a deliberate choice: statistical models are transparent and explainable, which matters enormously when the output influences real purchasing decisions.

Where the forecasting model runs

There is no separate 'AI server.' The forecasting math is plain Python code that runs as part of the same backend described in Section 1 - on Render, alongside everything else. When you open the AI Forecasting or Purchase Recommendations page, the backend runs these calculations on the spot and returns the result. Nothing is pre-trained or hosted separately.

How it actually decides what to forecast

For each product, the system first asks: *how much real sales history do we actually have for this exact SKU?* The model used depends entirely on the answer - it never applies the same method blindly to a SKU with three days of history and one with three years:

- **Fewer than 6 days of history** - the system explicitly says 'not enough data yet' and shows a zero forecast rather than inventing a number. This is a deliberate refusal to fabricate.
- **6-89 days** - uses a *simple moving average* (a plain recent-sales average). Too little history to trust anything fancier without overfitting noise.
- **90 days to 12 months** - uses a *weighted moving average*, which leans more heavily on the most recent weeks than older ones.
- **12+ months** - uses *Holt's linear trend method*, a well-known technique that can project a rising or falling trend forward - deliberately capped so a single unusual hot streak can't be extrapolated into an absurd number.

How the system checks its own accuracy

Every forecast is **backtested** before being shown: the model is temporarily hidden from the most recent couple of weeks of real sales, asked to predict them anyway, and then graded against what actually happened, using standard accuracy measures (MAE, RMSE, MAPE - different ways of measuring 'how far off was the guess'). That grade becomes the displayed **Risk Level** (Low/Medium/High). This is separate from **Confidence** (High/Medium/Low), which only reflects how much history exists. A SKU can have High confidence (lots of data) but Medium or High risk (that data happens to be volatile and hard to predict) - the two are shown independently on purpose, instead of collapsing them into one misleading score.

How a purchase quantity is actually calculated

The suggested order quantity for a SKU comes from one explicit, documented formula - never a hidden black box:

$$\text{Suggested Order} = \max(0, \text{Forecast Demand During Lead Time} + \text{Safety Stock} - \text{Current Stock} - \text{Incoming Stock})$$

- **Forecast Demand During Lead Time** = the SKU's forecast daily sales rate x the supplier's lead time (how many days it takes them to deliver, pulled from that supplier's own record, or a configurable default).
- **Safety Stock** = a buffer, calculated as the daily sales rate x a configurable number of 'safety' days (defaults to 14, adjustable in Settings).
- **Current Stock** = the live ledger total from Section 4.
- **Incoming Stock** = quantity already on order from open purchase orders for that SKU.

The result is then rounded up to the nearest multiple of that SKU's minimum order quantity, if one is set. Every recommendation stores every one of these numbers alongside itself, so 'why 200 units?' always has a concrete, inspectable answer on screen - never just a bare number to trust blindly.

Nothing here ever places an order automatically. Every recommendation sits as 'Pending' until a person explicitly accepts or dismisses it. The AI's job stops at decision support.

7. How Importing a Spreadsheet Actually Works

When a sales or purchases file is uploaded, the backend first reads only its column headers and compares them against known patterns for each record type - a file with a price/rate column and no cost column scores as 'Sales'; one with a cost/supplier column scores as 'Purchases,' and so on. This is why no manual 'what kind of file is this' step is needed anymore.

Every row is then individually validated - missing SKU, invalid date, negative quantity, a SKU that doesn't exist yet, a duplicate row within the same file - before anything touches the database. If every row checks out clean, the import commits immediately with no extra click. If anything is wrong, it stops and shows exactly which rows and why, and only the valid rows get imported once you confirm - invalid rows are always skipped, never guessed at or silently corrected.

8. Quick Reference

Live site	inventory-management-wheat-two.vercel.app
Backend health check	shoexpress-api.onrender.com/api/health - loading this should show <code>{"status":"ok"}</code> . Useful first check if something seems down.
Source code	github.com/deveshgoel02/Inventory-Management (private repository)
Full technical docs	Inside the repository: README.md (setup/deployment) and docs/business-rules.md (the exact formula behind every number the app shows).
If the site feels slow on first load	Almost certainly the Render free-tier wake-up delay (Section 3) - wait ~30-60 seconds; it will be fast again afterward.
If a stock number looks wrong	Open that SKU's page and read its ledger (Section 4) - it will show every transaction that built up to the current number.
Changing your password / profile	Sidebar - Profile. Any user can update their own name, email, and password; only an ADMIN account can change roles.

Prepared as a plain-language companion to the technical documentation already in the project repository.